

ACTIVIDAD Y TIPO: “Ejercicios PL-SQL”/AEF

LUGAR DE REALIZACIÓN: Centro docente/Casa

Unidad Didáctica	Criterios de Evaluación	Resultados de Aprendizaje
UD8	RA4.d) Se han diseñado guiones de sentencias para llevar a cabo tareas complejas.	RA4
UD8	RA4.g) Se han identificado los efectos de las distintas políticas de bloqueo de registros.	RA4
UD8	RA4.h) Se han adoptado medidas para mantener la integridad y consistencia de la información.	RA4
UD8	RA5.a) Se han identificado las diversas formas de automatizar tareas.	RA5
UD8	RA5.b) Se han reconocido los métodos de ejecución de guiones.	RA5
UD8	RA5.c) Se han identificado las herramientas disponibles para editar guiones.	RA5
UD8	RA5.d) Se han definido y utilizado guiones para automatizar tareas.	RA5
UD8	RA5.e) Se han utilizado estructuras de control de flujo.	RA5
UD8	RA5.f) Se ha hecho uso de las funciones proporcionadas por el sistema gestor.	RA5
UD8	RA5.g) Se han definido funciones de usuario.	RA5
UD8	RA5.h) Se han definido disparadores.	RA5
UD8	RA5.i) Se han utilizado cursores.	RA5

Objetivos Generales	Competencias Profesionales, Personales y Sociales
b) c) e) p) q) t) w)	c) e) f)

- **Individual/ Equipo:** Individual

-
- **Descripción de la actividad:** Realizar los ejercicios solicitados en cada grupo del enunciado en el momento en que las docentes lo requieran. Se entregaran los grupos por separado. En cada grupo indicar el número del ejercicio en comentario. Si alguno no se sabe resolver, dejarlo en blanco o indicar motivo en un comentario.
 - **Fecha y medio de entrega:** La fecha y medio de entrega se indicará en la publicación de la tarea en Classroom. Si se incumplen las especificaciones indicadas en la actividad tanto de contenido como de fecha, formato, nombre o forma de entrega, la actividad será calificada con un 0.
 - **Formato de entrega:** Entregar un archivo de nombre **GRUPO x.sql**, conteniendo en la primera línea, el nombre y apellidos del alumno comentado. Para cada ejercicio deberá comentarse el número. Si no se sabe resolver, se dejará en blanco o se pondrá la solución intentada, indicando en un comentario el problema encontrado en su intento de resolución.

GRUPO 1

1. Indicar qué hace el siguiente bloque PL/SQL.

```
SQL>DECLARE
2  num NUMBER;
3  BEGIN
4  SELECT COUNT(*) INTO num
5  FROM PRODUCTOS;
6  DBMS_OUTPUT.PUT_LINE(num);
7  END;
8 /
```

2. Introducir el bloque anterior desde SQL*Plus y guardarlo en un fichero.

3. Cargar y ejecutar el bloque de nuevo. (Comprobar que aparecen el resultado en pantalla).

4. Identificar en el ejemplo los siguientes elementos:

```
CREATE OR REPLACE
PROCEDURE verPrecio(codProducto NUMBER)
IS
  precio NUMBER;
BEGIN
  SELECT PRECIO_UNI INTO precio FROM PRODUCTOS
  WHERE COD_PRODUCTO = codProducto;
  DBMS_OUTPUT.PUT_LINE(precio);
END;
```

¿Cuál es la cabecera del procedimiento?.

¿Qué es *precio*?.

¿Qué es *codProducto*?.

¿Qué es *precio_uni*?.

¿Cuáles son los parámetros del procedimiento?.

¿Cuál es el nombre del procedimiento?

¿Dónde comienza el bloque?.

¿Qué hace la cláusula INTO?.

¿Por qué no tiene la cláusula DECLARE? ¿Qué tiene en su lugar?

5. Indicar los errores que aparecen en las siguientes instrucciones y la forma de corregirlos.

```
DECLARE
  Num1 NUMBER(8,2) := 0
  Num2 NUMBER(8,2) NOT NULL DEFAULT 0;
  Num3 NUMBER(8,2) NOT NULL;
  Cantidad INTEGER(3);
  Precio, Descuento NUMBER(6);
  Num4 Num1%ROWTYPE;
  Dto CONSTANT INTEGER;
BEGIN
  ...
END;
```

GRUPO 2

1. Escribir un procedimiento que reciba dos números y visualice su suma.
2. Codificar un procedimiento que reciba una cadena y la visualice al revés.
3. Escribir una función que reciba una fecha y devuelva el año, en número, correspondiente a esa fecha.
4. Escribir un bloque PL/SQL que haga uso de la función anterior.
5. Dado el siguiente procedimiento:

```
CREATE OR REPLACE PROCEDURE crearDepart (  
    numDept DEPART.DEPT_NO%TYPE,  
    nombreDept DEPART.DNOMBRE%TYPE DEFAULT 'PROVISIONAL',  
    locDept DEPART.LOC%TYPE DEFAULT 'PROVISIONAL')  
IS  
BEGIN  
    INSERT INTO DEPART  
        VALUES (numDept, nombreDept, locDept);  
END crearDepart;
```

Indicar cuáles de las siguientes llamadas son correctas y cuáles incorrectas, en este último caso escribir la llamada correcta usando la notación posicional (en los casos que se pueda):

crearDepart;	-- 1º
crearDepart(50);	-- 2º
crearDepart('COMPRAS');	-- 3º
crearDepart(50,'COMPRAS');	-- 4º
crearDepart('COMPRAS', 50);	-- 5º
crearDepart('COMPRAS', 'VALENCIA');	-- 6º
crearDepart(50, 'COMPRAS', 'VALENCIA');	-- 7º
crearDepart('COMPRAS', 50, 'VALENCIA');	-- 8º
crearDepart('VALENCIA', 'COMPRAS');	-- 9º
crearDepart('VALENCIA', 50);	-- 10º

6. Desarrollar una función que devuelva el número de años completos que hay entre dos fechas que se pasan como argumentos.
7. Escribir una función que, haciendo uso de la función anterior devuelva los trienios que hay entre dos fechas. (Un trienio son tres años completos).
8. Codificar un procedimiento que reciba una lista de hasta 5 números y visualice su suma.
9. Escribir una función que devuelva solamente caracteres alfabéticos sustituyendo cualquier otro carácter por blancos a partir de una cadena que se pasará en la llamada.
10. Implementar un procedimiento que reciba un importe y visualice el desglose del cambio en unidades monetarias de 1, 5, 10, 25, 50, 100, 200, 500, 1000, 2000, 5000 Ptas. en orden inverso al que aparecen aquí enumeradas.
11. Codificar un procedimiento que permita borrar un empleado cuyo número se pasará en la llamada.
12. Escribir un procedimiento que modifique la localidad de un departamento. El procedimiento recibirá como parámetros el número del departamento y la localidad nueva.
13. Visualizar todos los procedimientos y funciones del usuario, almacenados en la base de datos y su situación (valid o invalid).

GRUPO 3

1. Desarrollar un procedimiento que visualice el apellido y la fecha de alta de todos los empleados ordenados por apellido.
2. Codificar un procedimiento que muestre el nombre de cada departamento y el número de empleados que tiene.
3. Escribir un procedimiento que reciba una cadena y visualice el apellido y el número de empleado de todos los empleados cuyo apellido contenga la cadena especificada. Al finalizar visualizar el número de empleados mostrados.
4. Escribir un programa que visualice el apellido y el salario de los cinco empleados que tienen el salario más alto.
5. Codificar un programa que visualice los dos empleados que ganan menos de cada oficina.
6. Escribir un programa que muestre, en formato similar al proporcionado los siguientes datos:
 - Para cada empleado: apellido y salario.
 - Para cada departamento: Número de empleados y suma de los salarios del departamento.
 - Al final del listado: Número total de empleados y suma de todos los salarios.

```

CEREZO      *    318,500
MUÑOZ       *    169,000
REY         *    650,000
*** DEPTO: 10 NUM. EMPLEADOS: 3 SUM. SALARIOS:  1,137,500
ALONSO      *    143,000
FERNÁNDEZ   *    390,000
GIL         *    390,000
JIMÉNEZ     *    386,750
SÁNCHEZ     *    104,000
*** DEPTO: 20 NUM. EMPLEADOS: 5 SUM. SALARIOS:  1,413,750
ARROYO      *    208,000
JIMENO      *    123,500
MARTÍN      *    162,500
NEGRO       *    370,500
SALA        *    162,500
TOVAR       *    195,000
*** DEPTO: 30 NUM. EMPLEADOS: 6 SUM. SALARIOS:  1,222,000
***** NUMERO TOTAL EMPLEADOS: 14 TOTAL SALARIOS:  3,773,250

```

7. Desarrollar un procedimiento que permita insertar nuevos departamentos según las siguientes especificaciones:
 - Se pasará al procedimiento el nombre del departamento y la localidad.
 - El procedimiento insertará la fila nueva asignando como número de departamento la decena siguiente al número mayor de la tabla.
 - Se incluirá gestión de posibles errores.
8. Escribir un procedimiento que reciba todos los datos de un nuevo empleado procese la transacción de alta, gestionando posibles errores.
9. Codificar un procedimiento reciba como parámetros un número de departamento, un importe y un porcentaje; y suba el salario a todos los empleados del departamento indicado en la llamada. La subida será el porcentaje o el importe indicado en la llamada (el que sea más beneficioso para el empleado en cada caso empleado).
10. Escribir un procedimiento que suba el sueldo de todos los empleados que ganen menos que el salario medio de su oficina. La subida será del 50% de la diferencia entre el salario del empleado y la media de su oficina. Se deberá asegurar que la transacción no se quede a medias, y se gestionarán los posibles errores.
11. Diseñar una aplicación que simule un listado de liquidación de los empleados según las siguientes especificaciones:
 - El listado tendrá el siguiente formato para cada empleado:

Liquidación del empleado:.....(1) Dpto:.....(2) Oficio:.....(3)

Salario :.....(4)

Trienios :.....(5)

Comp. Responsabil :.....(6)

Comisión :.....(7)

Total :.....(8)

- Donde: **1 ,2, 3 y 4** Corresponden al apellido, departamento, oficio y salario del empleado.

5 Es el importe en concepto de trienios. Cada trienio son tres años completos desde la fecha de alta hasta la de emisión y supone 5000 Ptas.

6 Es el complemento por responsabilidad. Será de 10000Ptas por cada empleado que se encuentre directamente a cargo del empleado en cuestión.

7 Es la comisión. Los valores nulos serán sustituidos por ceros.

8 Suma de todos los conceptos anteriores.

- El listado irá ordenado por Apellido.

12. Crear la tabla LIQUIDACION con las columnas *apellido, departamento, oficio, salario, trienios, comp_responsabilidad, comisión y total*; y modificar la aplicación anterior para que en lugar de realizar el listado directamente en pantalla, guarde los datos en la tabla. Se controlarán todas las posibles incidencias que puedan ocurrir durante el proceso.

GRUPO 4. TRIGGERS

1. Construir un disparador de base de datos que permita auditar las operaciones de inserción o borrado de datos que se realicen en la tabla EMPLE según las siguientes especificaciones:

A. En primer lugar se creará desde SQL *Plus la tabla AUDITAREMPLE con la columna COL1 VARCHAR2(200).

B. Cuando se produzca cualquier manipulación se insertará una fila en dicha tabla que contendrá:

Fecha y hora

Número de empleado

Apellido

La operación de actualización: INSERCIÓN o BORRADO

2. Escribir un trigger que permita auditar las modificaciones en la tabla empleados insertado en la tabla AUDITAREMPLE los siguientes datos:

Fecha y hora

Número de empleado

Apellido

La operación de actualización: MODIFICACIÓN.

El valor anterior y el valor nuevo de cada columna modificada. (solo las columnas modificadas)

3. Escribir un disparador de base de datos que haga fallar cualquier operación de modificación del apellido o del número de un empleado, o que suponga una subida de sueldo superior al 10%.

4. Suponiendo que disponemos de la vista

CREATE VIEW VISTADPTOS AS

```
SELECT DEPART.DEPT_NO, DNOMBRE, LOC, COUNT(EMP_NO) TOT_EMPLE
FROM EMPLE, DEPART
WHERE EMPLE.DEPT_NO (+) = DEPART.DEPT_NO
GROUP BY DEPART.DEPT_NO, DNOMBRE, LOC;
```

Construir un disparador que permita realizar operaciones de actualización en la tabla DEPART a partir de la vista VISTADPTOS, de forma similar al ejemplo del trigger gestionVistaEmple. Se contemplarán las siguientes operaciones:

Insertar departamento.

Borrar departamento.

Modificar la localidad de un departamento.